

Report primo progetto di Metodi del calcolo scientifico

Daniele Besozzi 894998
Andrea Consonni 900116
Matteo Cervini 902225

19 maggio 2026

Indice

1	Presentazione degli ambienti	2
1.1	Matlab	2
1.1.1	Gestione della memoria di swap	2
1.2	C++	4
1.2.1	Eigen	4
1.2.2	CHOLMOD	4
1.2.3	Formato Matrix Market	5
1.2.4	Cross platform Make (CMake)	5
1.2.5	Installazione e compilazione su Windows	6
1.2.6	Installazione e compilazione su Linux	7
2	Tempo di esecuzione	8
2.1	Modalità di misurazione	8
2.2	Risultati	9
3	Utilizzo della memoria	10
3.1	Misura utilizzata	10
3.2	Profiling della memoria	10
4	Precisione delle soluzioni	12
5	Conclusioni	13

Capitolo 1

Presentazione degli ambienti

Le misure effettuate per questo progetto sono state eseguite su una macchina con le seguenti caratteristiche:

- Processore: AMD ryzen 7 5700G (8 core, 16 thread, 3.8 GHz)
- RAM: 32 GB DDR4 3200 MHz
- Sistemi operativi: Windows 11 25H2 e Linux Fedora 44
- Linguaggi di programmazione: C++ e Matlab
- Dischi: SSD NVMe

1.1 Matlab

Per quanto riguarda Matlab, abbiamo utilizzato la versione R2025b. Per la risoluzione dei sistemi di equazioni è stato utilizzato il comando `mldivide`, che implementa una serie di controlli per scegliere il metodo più adatto a seconda del tipo di matrice (densa, sparsa, simmetrica, ecc.). Questa libreria viene mantenuta e aggiornata regolarmente da MathWorks, ottimizzando i metodi di risoluzione per alcuni tipi di matrici e migliorando le prestazioni complessive. La documentazione ufficiale di Matlab fornisce i dettagli sulla selezione del metodo di risoluzione in base alle caratteristiche della matrice, dividendo in due macro categorie: matrici dense e matrici sparse. In particolare, in figura 1.1 possiamo vedere il processo di selezione del metodo di risoluzione per matrici sparse, che è quello che abbiamo utilizzato per i nostri benchmark.

1.1.1 Gestione della memoria di swap

Quando un processo finisce la memoria fisica (RAM) disponibile, il sistema operativo trasferisce temporaneamente parte della memoria allocata per quel processo su un supporto di memoria secondario, come un disco rigido o un'unità SSD. Le parti di memoria trasferite su disco vengono successivamente riportate in RAM quando il processo ne richiede l'utilizzo. Questo meccanismo, che prende il nome di **swapping**, permette al sistema operativo di eseguire più processi anche quando la quantità di memoria fisica libera è limitata. Tendenzialmente, la memoria trasferita durante lo swapping viene salvata o in file sul disco, chiamati comunemente **file di swap** (o file di paging su Windows), oppure in partizioni sul disco dedicate. Tuttavia, questa operazione comporta un peggioramento delle performance, dato che l'accesso ai dati su disco è estremamente più lento rispetto ad un accesso diretto alla RAM. Durante l'esecuzione del programma Matlab su entrambi i sistemi operativi, abbiamo riscontrato che esso era oggetto di swapping da parte di quest'ultimi durante la computazione delle matrici di dimensione maggiore. Per gestire questa problematica e mitigarne gli effetti sulle performance, abbiamo adottato le seguenti soluzioni:

- **Windows:** Windows gestisce autonomamente il proprio file di swap; quindi non sono state necessarie ulteriori azioni
- **Linux Fedora:** Fedora (e altre distribuzioni Linux) oltre al classico meccanismo di swap, utilizza un particolare sistema di swap chiamato **zram**. Questo sistema prevede l'allocazione dinamica di un "disco" compreso direttamente nella memoria RAM del sistema, permettendo quindi di effettuare

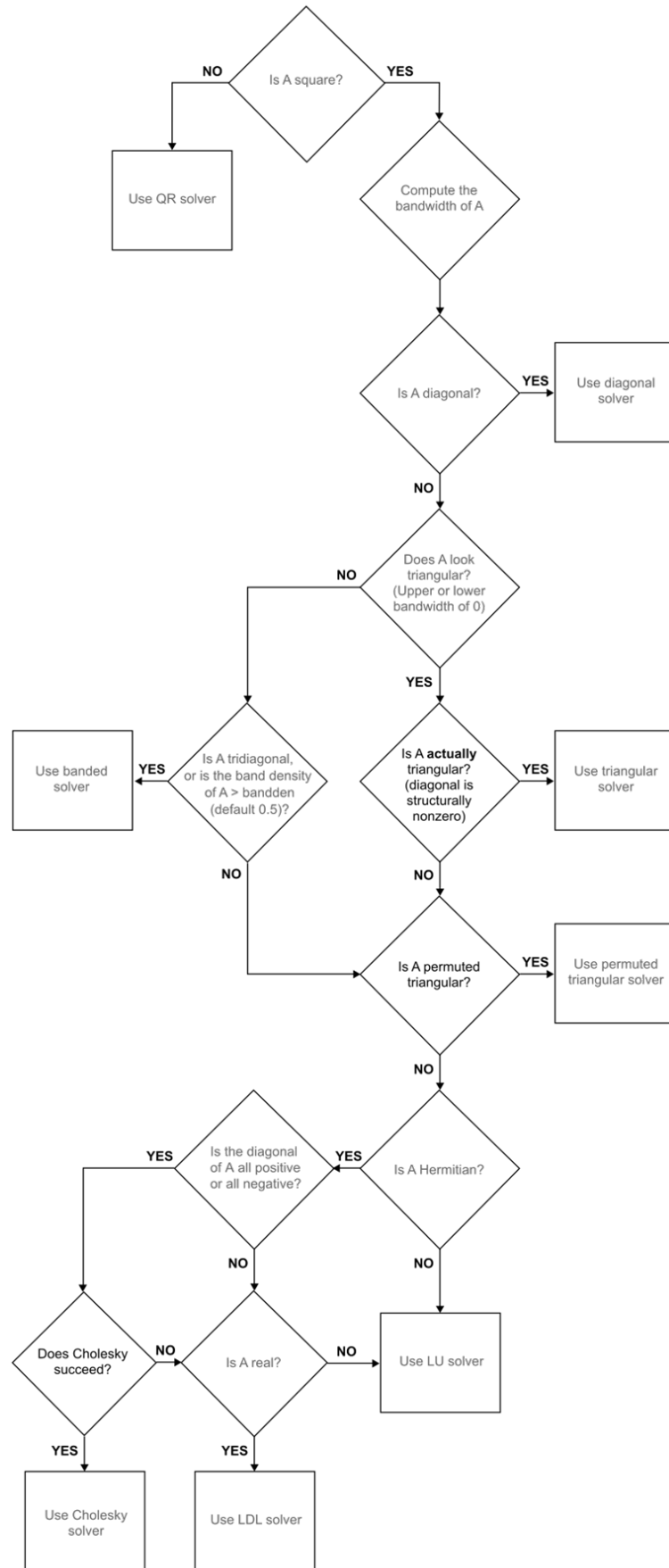


Figura 1.1: Scelta del metodo di risoluzione in Matlab per matrici sparse di `mldivide` [1].

lo swap più velocemente rispetto ad un swap effettuato su memoria secondaria. Essendo allocato in RAM, tuttavia, la sua dimensione non può essere troppo elevata. L’approccio utilizzato su Fedora è quindi stato il seguente:

- **zram**: La documentazione di Fedora [2] raccomanda di allocare una dimensione di zram pari al 50% della memoria RAM del sistema. Abbiamo quindi impostato una grandezza pari a 16 GB.
- **File di swap**: Poiché Matlab, durante la computazione delle matrici più grandi, alloca ben oltre 16 GB di RAM, abbiamo allocato un ulteriore file di swap su disco, con un grandezza di 32 GB, così che esso possa essere utilizzato quando la memoria in zram termina.

1.2 C++

Lo sviluppo del programma in C++ ha richiesto l’utilizzo di diverse librerie, con l’obiettivo di garantire un confronto equo con il programma sviluppato per Matlab. Questa sezione andrà ad illustrare le principali tecnologie e librerie adottate, descrivendone brevemente le caratteristiche, lo stato di manutenzione e la qualità della relativa documentazione.

1.2.1 Eigen

Eigen è una libreria open source per l’algebra lineare che offre una vasta gamma di classi e funzioni per la manipolazione di matrici e vettori, la rappresentazioni di diversi tipi di oggetti matematici e per la risoluzione di sistemi di equazioni lineari. La libreria è stata progettata per essere semplice da installare, efficiente e affidabile. **Eigen** permette inoltre di interfacciarsi con diversi moduli esterni per la risoluzione di sistemi di equazioni lineari, come, ad esempio, il pacchetto **SuiteSparse**, il quale è stato utilizzato per i benchmark presentati in questo report. In particolare, abbiamo utilizzato la libreria **CHOLMOD** di **SuiteSparse**, la quale offre un metodo di risoluzione tramite fattorizzazione di Cholesky più efficiente rispetto al risolutore standard con metodo di Cholesky offerto da **Eigen** [3]. L’interfaccia di **Eigen** per **CHOLMOD**, contenuta nel modulo **CholmodSupport** [4], permette di sfruttare le diverse funzionalità offerte dalla libreria. **Eigen** è attivamente supportata e mantenuta, con l’ultima versione stabile della libreria (5.0.0) rilasciata il 30 settembre 2025. La documentazione di **Eigen** è di ottima qualità e include sia tutorial introduttivi su installazione e utilizzo, sia sezioni più approfondite sul funzionamento interno della libreria e sulla sua integrazione con altre tecnologie e solver.

1.2.2 CHOLMOD

CHOLMOD è una libreria, facente parte del pacchetto **SuiteSparse**, che permette di svolgere efficientemente diverse operazioni su matrici sparse. In particolare, la libreria offre funzioni per la fattorizzazione di matrici sparse, simmetriche e definite positive tramite il metodo di Cholesky e per la risoluzione di sistemi lineari. La libreria è scritta nel linguaggio di programmazione C. **CHOLMOD** implementa due tipi di risolutori di sistemi di equazioni lineari sparsi che sfruttano questa decomposizione: uno **supernodale** e uno **non-supernodale** [5]. In particolare, abbiamo scelto di utilizzare l’implementazione **supernodale** dell’algoritmo di risoluzione, il quale è accessibile in **Eigen** costruendo un risolutore della classe **CholmodSupernodalLLT**[6]. Gli algoritmi supernodali si basano sulla decomposizione della matrice A in termini di blocchi di colonne, chiamati **supernodi**. L’algoritmo implementato da **CHOLMOD** è composto da tre fasi di calcolo principali:

1. La prima analizza la struttura della matrice e riordina le righe e le colonne per minimizzare il fill-in, ovvero l’introduzione di elementi non-nulli in L che non compaiono nella posizione corrispondente in A , dove L è la matrice triangolare inferiore ottenuta dalla decomposizione di Cholesky $A = LL^T$. Il riordinamento della matrice è un’importante fattore di performance, dato che con meno valori non nulli il numero di operazioni necessario e il peso in memoria diminuisce.
2. La seconda fase fattorizza la matrice applicando operazioni sui supernodi
3. La terza risolve due sistemi triangolari di equazioni: $Ly = b$ e $L^T x = y$ nel caso della fattorizzazione di Cholesky.

Tra le tre fasi, quella di fattorizzazione rappresenta la parte computazionalmente più onerosa del risolutore implementato da CHOLMOD [7]. Per sfruttare al meglio le moderne architetture delle CPU multicore, CHOLMOD utilizza la libreria **Open MultiProcessing (OpenMP)** [8] per implementare la parallelizzazione di alcune delle operazioni necessarie alla risoluzione del sistema di equazioni lineari, permettendo quindi di sfruttare al meglio la CPU e, agli scopi del benchmark, ottenere un confronto più equo con Matlab. Infine, CHOLMOD dipende inoltre da diverse librerie per effettuare i calcoli necessari alla decomposizione e alla risoluzione del sistema di equazioni lineari:

- **Librerie per l'ordinamento della matrice:** Per effettuare l'ordinamento della matrice nella prima fase dell'algoritmo di risoluzione, CHOLMOD può utilizzare diversi algoritmi, come *Approximate Minimum Degree ordering* e METIS [7]. Per questo motivo, CHOLMOD dipende dalle librerie di SuiteSparse AMD, CAMD, COLAMD, CCOLAMD e METIS.
- **Basic Linear Algebra Subroutines (BLAS):** BLAS [9] è un insieme di specifiche open per l'implementazione di funzioni a basso livello per svolgere le comuni operazioni richieste per l'algebra lineare. Esistono diverse implementazioni delle funzioni descritte da BLAS, tra cui **OpenBLAS** e **FlexiBLAS**, entrambe utilizzate per i benchmark presentati in questo report.
- **Linear Algebra PACKage (LAPACK):** LAPACK [10] è una libreria che offre diverse funzioni per la risoluzione di sistemi lineari. La libreria dipende dalla presenza di un'implementazione di BLAS ed è scritta in modo tale che la computazione sia svolta effettuando il più possibile chiamate alle funzioni offerte dall'implementazione di BLAS sottostante.

SuiteSparse e CHOLMOD sono librerie attivamente mantenute. In particolare, CHOLMOD dispone di una documentazione tecnica ampia e dettagliata [8], la quale copre in modo esaustivo sia gli aspetti implementativi sia le modalità d'uso della libreria. Inoltre, CHOLMOD viene descritta dai suoi autori in [5].

1.2.3 Formato Matrix Market

Per mantenere il programma scritto in C++ completamente indipendente da tecnologie o formati proprietari, si è deciso di utilizzare la rappresentazione delle matrici di test nel formato **Matrix Market Exchange Format** [11], un formato di file per la rappresentazione di matrici creato dal **National Institute for Standards and Technology** (NIST) e scritto completamente in caratteri ASCII. Questo formato prevede due tipi diversi di rappresentazione per le matrici:

- **Formato a Coordinate:** Questo formato è adatto per rappresentare generiche matrici sparse. Vengono fornite solamente le entrate non nulle e le loro coordinate all'interno della matrice.
- **Formato ad array:** Questo formato è adatto per rappresentare generiche matrici dense. Vengono fornite tutte le entrate, elencate per colonne.

Per permettere un caricamento rapido delle matrici in questo formato, si è deciso di utilizzare la libreria open source **fast_matrix_market**[12], la quale mette a disposizione un modulo veloce e parallelizzato per caricare le matrici sparse direttamente nei tipi di oggetti definiti da Eigen. **fast_matrix_market** non è più attivamente sviluppato (l'ultima commit al progetto risale a 2 anni prima della stesura di questo report). Inoltre, la libreria non dispone di una documentazione ufficiale, preferendo invece l'utilizzo di brevi file README che descrivono le funzionalità disponibili, le scelte implementative adottate e alcuni esempi d'uso per sfruttarne al meglio le funzionalità.

1.2.4 Cross platform Make (CMake)

Cross platform Make [13], abbreviato in CMake, è uno strumento di automazione che permette di gestire in maniera semplice il processo di compilazione e build di un progetto. CMake è largamente usato per i linguaggi C e C++, tuttavia esso può essere usato per la gestione di progetti scritti anche in altri linguaggi di programmazione. Il tipico uso di CMake ruota intorno ad uno più file **CMakeList.txt**, comunemente chiamati "list files". In un progetto software gestito con CMake, avremo un list file per ogni cartella per la quale vogliamo specificare le modalità di gestione dei file e quali sono le operazioni necessarie da svolgere al suo interno. Nel progetto C++ sviluppato per questo benchmark, abbiamo utilizzato un singolo list file per la gestione di tutto il progetto, nel quale vengono specificate le seguenti operazioni da svolgere ogni volta che il progetto viene compilato:

1. Se non sono presenti, scaricare le matrici di test in formato .mtx
2. Importare tutte le dipendenze necessarie al funzionamento del progetto
3. Specificare come le dipendenze debbano essere linkate al progetto e quali solo le flag di ottimizzazione che il compilatore deve utilizzare
4. Disabilitare le asserzioni interne ad **Eigen** (linee di codice utilizzate per effettuare dei test interni durante l'esecuzione), per migliorare la velocità di esecuzione del programma, ed indicare alla libreria di utilizzare l'implementazione di **BLAS** presente nel sistema
5. Rendere disponibili al programma i percorsi delle cartelle rilevanti per l'esecuzione del progetto, per evitare di indicarli esplicitamente per ogni sistema operativo

CMake è un software costantemente aggiornato e supportato da una documentazione completa e ben strutturata. La documentazione mette a disposizione sia guide introduttive per familiarizzare con lo strumento, sia sezioni di riferimento dettagliate per tutti i comandi offerti.

1.2.5 Installazione e compilazione su Windows

Per l'esecuzione e la gestione dello sviluppo del programma su piattaforma Windows, abbiamo deciso di utilizzare **Visual Studio 2026**; il quale viene distribuito insieme al compilatore per C/C++ **Microsoft Visual C++** (MSVC) e ad una serie di tool utili per lo sviluppo. In particolare, il **Windows Software Development Kit** (SDK) [14] mette a disposizione un insieme di librerie per lo sviluppo di applicazioni su piattaforma Windows; tra cui le librerie `windows.h`, la quale contiene tutte le dichiarazioni e definizioni necessarie ad utilizzare le librerie offerte dall'SDK, e `Psapi.h` (Process status API) [15], le cui funzioni permettono di accedere alle informazioni relative ai processi in esecuzione, come le informazioni legate all'utilizzo della memoria. L'installazione delle librerie e delle dipendenze necessarie ad eseguire il programma ha invece richiesto diversi passaggi aggiuntivi, descritti nelle sezioni seguenti.

Eigen

Eigen può essere facilmente installato su Windows nella sua ultima versione utilizzando il gestore di pacchetti open source `vcpkg` [16].

SuiteSparse e OpenBLAS

Purtroppo, non tutti i pacchetti disponibili su `vcpkg` sono compilati e gestiti in maniera corretta. È questo il caso dei pacchetti **SuiteSparse** e **OpenBLAS**, le quali versioni disponibili su `vcpkg` vengono installate senza il supporto al multithreading. Abbiamo quindi deciso di effettuare la compilazione e l'installazione manuale di entrambe le librerie. Per quanto riguarda **OpenBLAS**, la documentazione ufficiale [17] indica come compilare ed installare correttamente la libreria, inclusi i pacchetti necessari per abilitare l'utilizzo del multithreading (**OpenMP**). Nel caso di **SuiteSparse**, invece, abbiamo deciso di utilizzare le istruzioni presenti nella repository `suitesparse-metis-for-windows` [18], la quale mette a disposizione una procedura di installazione, supportata da programmi ausiliari, più semplice rispetto a quella presente sulla repository ufficiale del progetto [19].

Flag di ottimizzazione

Durante il processo di compilazione, abbiamo indicato a MSVC di utilizzare le seguenti flag di ottimizzazione:

- `/arch:AVX2`: Abilita Intel Advanced Vector Extension 2 [20], un'estensione delle istruzioni assembly per architetture x86 (la stessa architettura del processore usato per questi benchmark). AVX2 può aumentare, in alcuni casi, le performance nei calcoli con numeri a virgola mobile ed incrementare il parallelismo del programma
- `/openmp`: Abilita l'utilizzo di **OpenMP** per il programma

1.2.6 Installazione e compilazione su Linux

Per l'esecuzione e la gestione dello sviluppo del programma su Linux Fedora, abbiamo deciso di utilizzare **CLion**, sviluppato da JetBrains. Abbiamo inoltre deciso di utilizzare il compilatore **g++**, sviluppato dal progetto **GNU** e disponibile in tutte le principali distribuzioni Linux.

Installazione delle librerie

Tutte le librerie necessarie ad eseguire il progetto sono state facilmente reperibili utilizzando il package manager di Fedora **dnf**. In particolare, la documentazione di **OpenBLAS** [17] indica di utilizzare congiuntamente ad essa, su Fedora, la libreria wrapper **FlexiBLAS**[21], la quale permette di cambiare facilmente l'implementazione di **BLAS** che si vuole utilizzare. L'abilitazione del multithreading, quindi, ha richiesto solamente di indicare a **FlexiBLAS** di utilizzare l'implementazione fornita da **OpenBLAS** con **OpenMP**.

Flag di ottimizzazione

Come per **MSVC**, durante il processo di compilazione abbiamo indicato a **g++** di utilizzare le seguenti flag di ottimizzazione:

- **-O2**: Indica al compilatore di effettuare tutte le operazioni di ottimizzazione disponibili, tranne quelle che implicano un compromesso tra spazio e velocità [22].
- **-march=native**: Indica al compilatore di ottimizzare il codice generato per l'architettura della CPU corrente [23]
- **-DNDEBUG**: Indica al compilatore di disattivare le asserzioni interne alle librerie del programma, in modo da aumentarne le performance. Questa flag è in realtà formata da due parti:
 - **-D**: Indica al compilatore la definizione di una macro [24]
 - **NDEBUG**: Indica la disattivazione delle asserzioni [25]
- **-fopenmp**: Abilita l'utilizzo di **OpenMP** per il programma

Capitolo 2

Tempo di esecuzione

2.1 Modalità di misurazione

Per effettuare le misurazioni sul tempo di risoluzione del sistema, abbiamo utilizzato le funzionalità offerte nativamente dai rispettivi linguaggi:

- **Matlab**: abbiamo utilizzato i comandi `tic` e `toc` per rispettivamente far partire il timer e fermarlo.
- **C++**: per effettuare la misurazione, il programma C++ calcola la differenza di tempo da poco dopo aver letto la matrice in memoria a subito dopo la risoluzione del sistema. Per ottenere i due istanti di tempo necessari al calcolo, abbiamo utilizzato la funzione `std::chrono::high_resolution_clock::now()` della libreria `chrono` del linguaggio.

```
105 // Start the timer to measure total resolve time
106 auto start = std::chrono::high_resolution_clock::now();
107
108 // Get current resident set size after reading the matrix
109 const double mem_after_reading = get_current_memory_usage();
110
111 // Create xe vector of ones
112 Eigen::VectorXd xe = Eigen::VectorXd::Ones(A.rows());
113
114 // Create vector b as b = A * xe
115 Eigen::VectorXd b = A * xe;
116
117 // Initialize the Sparse Linear Matrix solver for applying Cholesky; force SuperNodal to
    ensure multithreading
118 Eigen::CholmodSupernodalLLT<Eigen::SparseMatrix<double>> solver;
119
120 // Compute the Cholesky factorization
121 solver.compute(A);
122
123 // Check if the Cholesky factorization has converged, thus if the matrix is definite
    positive
124 if (solver.info() != Eigen::Success) {
125     std::cerr << "Decomposition failed for: " << matrix_file << std::endl;
126     throw std::logic_error("Matrix is not definite positive!");
127 }
128
129 // Solve the linear system Ax = b
130 Eigen::VectorXd x = solver.solve(b);
131
132 // Stop the timer
133 auto end = std::chrono::high_resolution_clock::now();
```

Listing 2.1: Porzione del codice C++ in cui si calcola il tempo di esecuzione

```
66 % computation
67 tic
68 xe = ones(n,1);
69 b = A * xe;
70 x = A \ b;
71 relerr = norm(x - xe) / norm(xe);
72 solve_time = toc;
```

Listing 2.2: Porzione del codice Matlab in cui si calcola il tempo di esecuzione

2.2 Risultati

Capitolo 3

Utilizzo della memoria

3.1 Misura utilizzata

Poiché Linux e Windows offrono statistiche diverse riguardanti la memoria allocata per un processo, è stato necessario utilizzare due metriche di memoria diverse:

- **Private Bytes:**

3.2 Profiling della memoria

Per effettuare le misurazioni sullo spazio occupato in memoria dei due programmi sviluppati, è stato necessario scrivere due ulteriori programmi distinti, uno per Windows e uno per Linux, per il campionamento della memoria occupata da un determinato processo. Questa necessità nasce da problematiche legate sia a Matlab che a C++. Per quanto riguarda Matlab, la pressoché inesistente documentazione sul profiler integrato nell'ambiente di sviluppo [26] rende particolarmente arduo comprendere appieno la misurazione ottenuta. Per C++, invece, non abbiamo trovato un profiler agnostico al sistema operativo che indicasse la quantità di memoria utilizzata per ogni chiamata ad una certa funzione (nel nostro caso, la funzione `calculate_benchmark()`). Di seguito, riportiamo i due programmi scritti. I linguaggi utilizzati sono **Powershell** per Windows e **Bash** per Linux.

```
1 param( # Line command params
2     [int]$targetPid,    # PID of the process to monitor
3     [string]$outfile    # output file where memory values will be saved
4 )
5 # Write header line to the output file
6 "mem_kb" | Out-File $outfile -Encoding ascii
7 # Loop while the target process is still running
8 while (Get-Process -Id $targetPid -ErrorAction SilentlyContinue) {
9     # Get the process object
10    $p = Get-Process -Id $targetPid
11    # WorkingSet64 is in bytes, convert to KB
12    $memKB = [math]::Round($p.WorkingSet64 / 1KB)
13    # Append the memory value to the output file
14    "$memKB" | Out-File $outfile -Append -Encoding ascii
15    # Wait 10 milliseconds before next sample
16    Start-Sleep -Milliseconds 10
17 }
```

Listing 3.1: Sampler scritto per Windows in Powershell

```

1  #!/bin/bash
2  PID=$1    # Process ID passed as first argument
3  OUT=$2    # Output file passed as second argument
4  # Write header to output file
5  echo "mem_kb" > $OUT
6  # Loop while the process is still running
7  while kill -0 $PID 2>/dev/null; do
8      # Get resident set size (RSS) in KB for the process
9      MEM=$(ps -o rss= -p $PID)
10     # Append memory usage value to the output file
11     echo "$MEM" >> $OUT
12     # Sleep for 0.01 seconds before next sample
13     sleep 0.01
14 done

```

Listing 3.2: Sampler scritto per Linux in Powershell

Capitolo 4

Precisione delle soluzioni

Capitolo 5

Conclusioni

Bibliografia

- [1] *mldivide - Solve systems of linear equations $Ax = B$ for x - MATLAB*. visitato il giorno 14 mag. 2026. indirizzo: <https://it.mathworks.com/help/matlab/ref/double.mldivide.html>
- [2] Fedora Project, *SwapOnZRAM documentation*, 2020. visitato il giorno 18 mag. 2026. indirizzo: <https://www.fedoraproject.org/wiki/Changes/SwapOnZRAM>
- [3] *Eigen::SimplicialLLT Class Template Reference*. visitato il giorno 16 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1SimplicialLLT.html
- [4] *Eigen: CholmodSupport module*. visitato il giorno 15 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-5.0/group__CholmodSupport__Module.html
- [5] Y. Chen, T. A. Davis, W. W. Hager e S. Rajamanickam, «Algorithm 887: CHOLMOD, Supernodal Sparse Cholesky Factorization and Update/Downdate,» *ACM Transactions on Mathematical Software*, vol. 35, n. 3, pp. 1–14, ott. 2008, ISSN: 0098-3500, 1557-7295. DOI: 10.1145/1391989.1391995 visitato il giorno 15 mag. 2026. indirizzo: <https://dl.acm.org/doi/10.1145/1391989.1391995>
- [6] *Eigen::CholmodSupernodalLLT Class Template Reference*. visitato il giorno 15 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-5.0/classEigen_1_1CholmodSupernodalLLT.html
- [7] V. Le Fèvre, T. Usui e M. Casas, «A Selective Nesting Approach for the Sparse Multi-threaded Cholesky Factorization,» in *2022 IEEE/ACM 7th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*, nov. 2022, pp. 1–9. DOI: 10.1109/ESPM256814.2022.00006 visitato il giorno 15 mag. 2026. indirizzo: <https://ieeexplore.ieee.org/document/10029458>
- [8] T. Davis, *CHOLMOD User guide*, lug. 2025. visitato il giorno 17 mag. 2026. indirizzo: https://github.com/DrTimothyAldenDavis/SuiteSparse/blob/dev/CHOLMOD/Doc/CHOLMOD_UserGuide.pdf
- [9] *BLAS (Basic Linear Algebra Subprograms)*. visitato il giorno 16 mag. 2026. indirizzo: <https://www.netlib.org/blas/>
- [10] *LAPACK-Linear Algebra PACKage*. visitato il giorno 16 mag. 2026. indirizzo: <https://www.netlib.org/lapack/>
- [11] *Matrix Market: File Formats*. visitato il giorno 17 mag. 2026. indirizzo: <https://math.nist.gov/MatrixMarket/formats.html>
- [12] A. Lugowski, *fast_matrix_market: Fast and Full-Featured Matrix Market I/O Library*, gen. 2023. DOI: 10.5281/zenodo.10223767 visitato il giorno 17 mag. 2026. indirizzo: https://github.com/alugowski/fast_matrix_market
- [13] Kitware, Inc., *CMake Reference Documentation*, 2026. visitato il giorno 17 mag. 2026. indirizzo: <https://cmake.org/cmake/help/latest/>
- [14] Microsoft. «Windows SDK overview - Windows apps,» visitato il giorno 19 mag. 2026. indirizzo: <https://learn.microsoft.com/it-it/windows/apps/windows-sdk/>
- [15] Microsoft. «Process Status API (PSAPI) - Win32 apps,» visitato il giorno 19 mag. 2026. indirizzo: https://learn.microsoft.com/en-us/windows/win32/api/_psapi/
- [16] Microsoft. «vcpkg documentation,» visitato il giorno 19 mag. 2026. indirizzo: <https://learn.microsoft.com/en-us/vcpkg/>
- [17] OpenBLAS Contributors. «Install OpenBLAS,» visitato il giorno 19 mag. 2026. indirizzo: <https://github.com/OpenMathLib/OpenBLAS/blob/develop/docs/install.md>
- [18] José Luis Blanco-Claraco. «SuiteSparse + METIS for Windows,» visitato il giorno 19 mag. 2026. indirizzo: <https://github.com/jlblancoc/suitesparse-metis-for-windows>

- [19] Timothy A. Davis. «SuiteSparse: A Suite of Sparse Matrix Software,» visitato il giorno 19 mag. 2026. indirizzo: <https://github.com/DrTimothyAldenDavis/SuiteSparse>
- [20] P. Gepner, «Using AVX2 Instruction Set to Increase Performance of High Performance Computing Code,» *Computing and Informatics*, vol. 36, pp. 1001–1018, gen. 2017. DOI: 10.4149/cai_2017_5_1001
- [21] Martin Köhler, Markus Mützel. «FlexiBLAS: Flexible BLAS Library,» visitato il giorno 19 mag. 2026. indirizzo: <https://github.com/mpimd-csc/flexiblas>
- [22] Free Software Foundation. «GCC Optimization Options.» Accessed: 2026-05-19. indirizzo: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- [23] Free Software Foundation. «AArch64 Options: -march and -mcpu Feature Modifiers.» Accessed: 2026-05-19. indirizzo: https://gcc.gnu.org/onlinedocs/gcc/AArch64-Options.html#g_t-march-and--mcpu-Feature-Modifiers
- [24] Free Software Foundation. «Preprocessor Options: -D.» Accessed: 2026-05-19. indirizzo: <https://gcc.gnu.org/onlinedocs/gcc/Preprocessor-Options.html#index-D-1>
- [25] ISO C++ Standard Library. «assert macro (cassert / assert.h).» Accessed: 2026-05-19. indirizzo: <https://en.cppreference.com/w/cpp/error/assert>
- [26] MathWorks, *Profile Your Code to Improve Performance*, https://www.mathworks.com/help/matlab/matlab_prog/profiling-for-improving-performance.html, Accessed: 2026-05-19, 2025.